



Assignment: Event-Driven System with RabbitMQ + Onion Architecture (Spring Boot)

Goal

Build a distributed system using:

- **Spring Boot**
- **RabbitMQ**
- **Onion Architecture**

Focus on:

- clean architecture separation
 - event-driven communication
 - no direct service-to-service REST calls (except Auth login flow if needed)
-



Architecture Requirement (Mandatory)

Each service must follow **Onion Architecture**:

- **Domain Layer** → business logic only
- **Application Layer** → use cases / services
- **Infrastructure Layer** → DB, RabbitMQ, external tools
- **Presentation Layer** → REST controllers

No framework logic inside Domain layer.



Services (6 total)

1. Auth Service

Responsible for:

- User registration
- Login (JWT generation)
- Validating users
- Publishing event: `UserRegistered`

Events:

- `user.registered`
-

2. Order Service

- Creates orders via REST API
- Publishes: `OrderCreated`

Events:

- `order.created`
-

3. Payment Service

- Listens to `orderCreated`
 - Processes mock payment
 - Publishes:
 - `PaymentCompleted`
 - `PaymentFailed`
-

4. Inventory Service

- Listens to `orderCreated`
 - Checks stock availability
 - Publishes:
 - `StockReserved`
 - `StockFailed`
-

5. Shipping Service

- Listens to:
 - `PaymentCompleted`
 - `StockReserved`
 - Creates shipment record
 - Publishes:
 - `ShipmentCreated`
-

6. Notification Service

- Listens to ALL events:
 - `UserRegistered`
 - `OrderCreated`
 - `PaymentCompleted/Failed`
 - `StockReserved/Failed`
 - `ShipmentCreated`
 - Logs or prints notifications (no real email required)
-

RabbitMQ Setup

Exchange:

`app.exchange`

Routing Keys:

- `user.registered`
- `order.created`

- `payment.completed`
- `payment.failed`
- `stock.reserved`
- `stock.failed`
- `shipment.created`

Use:

- **Topic exchange recommended** (cleaner scaling)
-

System Flow (Simple View)

1. User registers → Auth Service
 2. Auth publishes `user.registered`
 3. User creates order → Order Service
 4. Order publishes `order.created`
 5. Payment + Inventory react in parallel
 6. If success → Shipping triggers
 7. Notification Service listens to everything
-

Mandatory Rules

- Each service must be independent Spring Boot app
 - Each service has its own database (H2 or PostgreSQL)
 - No direct REST calls between services (except Auth login usage)
 - Communication **ONLY** through RabbitMQ events
 - Proper Onion Architecture enforced everywhere
-

★ Bonus (Optional)

- JWT validation middleware in other services
- Retry + Dead Letter Queue (DLQ)
- Docker Compose for full system
- Central logging (ELK or simple file logs)
- Swagger for each service